

A Flexible Hardware Design Tool for Fast Fourier and Number-Theoretic Transformation Architectures

Florian Krieger[✉], Florian Hirner, Ahmet Can Mert, and Sujoy Sinha Roy[✉], *Senior Member, IEEE*

Abstract—Fully homomorphic encryption (FHE) and post-quantum cryptography (PQC) involve polynomial multiplications, which are a common performance bottleneck. To resolve this bottleneck, polynomial multiplications are often accelerated in hardware using the number-theoretic transformation (NTT) or the fast Fourier transformation (FFT). In particular, NTT operates over modular rings while FFT operates over complex numbers. NTT and FFT are widely deployed in applications with diverse parameter sets, leading to long design times for hardware accelerators. Existing hardware generation tools have limited functionality since they do not support generic on-the-fly twiddle factor generation (TFG) or different memory-related optimizations. This article improves the hardware design process and presents a generic and flexible tool to generate FFT and NTT architectures. In contrast to prior work, we combine on-the-fly TFG and stall-free memory accesses. Moreover, we enhance hardware design flexibility through memory-optimized (MemOpt) or routing-optimized (RoutOpt) design strategies. While our MemOpt strategy minimizes twiddle factors in ROM, our RoutOpt strategy allows significantly higher clock frequencies on FPGAs. These optimization strategies allow effective customization of NTT/FFT architectures, spanning from low-end PQC to high-end FHE accelerators. Compared to existing works, we reach up to 15.9× lower latency and up to 7.4× improved ATP for FFT applications such as the Falcon signature scheme. Considering other NTT tools, we decrease latency by up to 1.8× and 2× for PQC and FHE parameter sets, respectively.

Index Terms—Fast Fourier transformation (FFT), fully homomorphic encryption (FHE), hardware design tool, number-theoretic transformation (NTT), twiddle factor generation (TFG).

I. INTRODUCTION

CONTEMPORARY research dedicates substantial effort to the implementation of modern cryptographic schemes. Many of those schemes, including fully homomorphic encryption (FHE) [1], [2] and post-quantum cryptography (PQC) [3], [4], [5], rely on lattice problems, making the involved large-scale polynomial multiplications a considerable performance bottleneck. A highly relevant method to accelerate polynomial multiplication is the number-theoretic transformation (NTT), which reduces the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$, where N denotes the polynomial size [6].

Received 16 December 2024; revised 8 May 2025 and 24 June 2025; accepted 30 July 2025. Date of publication 4 August 2025; date of current version 23 April 2026. This work was supported in part by the State Government of Styria, Austria—Department Zukunftsfonds Steiermark. This article was recommended by Associate Editor Y. Lyu. (Corresponding author: Florian Krieger.)

The authors are with the Institute of Information Security, Technische Universität Graz, 8010 Graz, Austria (e-mail: florian.krieger@tugraz.at).
Digital Object Identifier 10.1109/TCAD.2025.3595834

The NTT typically operates over finite fields. However, some cryptographic applications, such as the PQ-signature scheme Falcon [4] or the FHE scheme CKKS [1], rely on the fast Fourier transformation (FFT), which operates over the complex numbers $\mathbb{C}$. Hence, the FFT requires a floating-point unit, whereas the NTT needs efficient modular reduction. Although FFT and NTT differ in their underlying arithmetic, they are closely related in the computation flow and algorithmic complexity. While NTT and FFT are widely used in cryptographic schemes, their hardware implementation poses a significant challenge. The wide range of FFT and NTT application scenarios—from low-end PQC designs to high-end FHE accelerators—requires versatile configurations. These configuration possibilities in algorithmic aspects (decimation method [6], negative-wrapped convolution (NWC) [7], etc.) and in hardware aspects (area consumption, performance, etc.) often lead to dedicated hardware architectures tailored to only one application scenario [8], [9], [10]. Extending these dedicated designs to other application scenarios is challenging and involves long design times and engineering effort.

To reduce the hardware design effort, research has proposed different NTT hardware design tools. One example of prior research is the work [11], where a flexible hardware generator is presented. The generator produces NTT architectures depending on polynomial degree N , modulus size $\log(q)$, and number of processing elements (PEs) n_{PE} . The main contribution of [11] is the scalable and conflict-free memory access pattern for in-place NTTs, which avoids pipeline stalls. Similarly, the work [12] supports different parameter sets at synthesis time, and even allows to change the polynomial degree N during runtime.

The very recent work [13] supports different NTT architectures and an HLS-based design space exploration. There also exist works proposing 2-D arrays of PEs [14]. In contrast to 1-D PE layouts, 2-D arrays reduce the overall number of memory accesses and, hence, lower the energy consumed in NTT transformations. Other works [15] reuse the stored twiddle factors (TFs). This measure lowers the ROM consumption since fewer TFs need to be kept in on-chip memory.

However, we note that all these prior tools for hardware generation rely on stored TFs, thus requiring large on-chip memory. This especially applies to high-degree polynomials as used in FHE. In contrast to storing, TFs can also be generated on the fly to lower memory consumption. Yet, supporting TF generation (TFG) in hardware design tools is challenging since conflict-free memory access patterns and a linear TFG order must be combined generically.

Our conference paper presented in [16] solves this issue and offers a hardware generation tool called OpenNTT. The OpenNTT tool from [16] allows for efficiently generating NTT hardware accelerators with TFG in a fully automated procedure. Yet, in this article, we extend the conference paper [16] and propose two design-time optimizations. The first optimization—called memory-optimized (MemOpt)—minimizes the TFs needed in ROM. Second, we propose routing-optimized (RoutOpt) to achieve higher clock frequencies in hardware at the cost of slightly increased ROM consumption. Thereby, we allow design-time flexibility between performance and resource consumption.

Moreover, the focus of the conference paper was on NTT acceleration in FHE applications. In this article, we extend to PQC applications and introduce support for the floating-point-based FFT. We present detailed benchmarks of various configurations and compare the results to related work in the field. Our contributions can be summarized as follows.

- 1) We present a conflict-free and generic processing order for efficient TFG. This processing order is applied to NTT and FFT transformations, which are both supported by our hardware generation tool.
- 2) We propose two design-time optimizations. First, our RoutOpt strategy elevates the frequency of FFT/NTT designs by improving routability at the cost of slightly higher twiddle memory consumption. Second, our MemOpt option minimizes the TF memory consumption but has a lower frequency. Thus, the generated hardware designs easily allow an area-performance tradeoff.
- 3) We give a detailed discussion of benchmarks for FFT and NTT architectures in the field of PQC and FHE parameter sets. Moreover, we open-source our OpenNTT tool in [17] to support research in the field.

Outline: Section II introduces notation and background. Section III presents our execution flow, RoutOpt, and MemOpt strategies. Section IV presents the flexible OpenNTT hardware architecture and the tooling workflow. Sections V and VI give implementation results and comparisons, respectively. Finally, Section VII presents a discussion and Section VIII concludes this article.

II. BACKGROUND AND NOTATION

We denote the ring of integers modulo a prime q as $\mathbb{Z}_q$. The polynomial ring $\mathcal{R}_q$ is defined as $\mathbb{Z}_q[x]/(x^N + 1)$, consisting of polynomials with coefficients in $\mathbb{Z}_q$, reduced modulo the irreducible polynomial $x^N + 1$. Here, N is restricted to powers of 2. We use lowercase letters in normal font ($a \in \mathbb{Z}_q$ or $b \in \mathbb{C}$) and bold font ($\mathbf{a} \in \mathcal{R}_q$) to distinguish between scalars and polynomials, respectively. The i th coefficient of a polynomial $\mathbf{a}$ is denoted as a_i or $\mathbf{a}[i]$. A polynomial $\mathbf{a}$ in coefficient representation can therefore, be expressed as $\mathbf{a} = \sum_{i=0}^{N-1} x^i a_i$. A polynomial $\mathbf{a} \in \mathcal{R}_q$ in its *evaluation* representation is denoted as $\hat{\mathbf{a}}$. We use $\cdot$, $\times$, and $\odot$ to distinguish between integer, polynomial, and coefficient-wise multiplication, respectively.

A. Number-Theoretic Transformation

The NTT is the discrete Fourier transformation over the polynomial ring $\mathcal{R}_q$. The NTT maps a polynomial $\mathbf{a}$ in its coefficient representation to its evaluation representation,

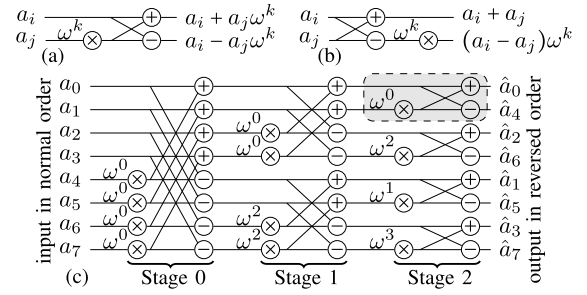


Fig. 1. (a) CT butterfly and (b) GS butterfly [16]. (c) Data flow for $N = 8$ point NR DIT transformation with CT butterfly highlighted.

denoted as $\hat{\mathbf{a}} = \text{NTT}(\mathbf{a})$. The NTT is defined as $\hat{a}_k = \sum_{i=0}^{N-1} a_i \omega^{ik} \bmod q$, where ω is an N th primitive root of unity in $\mathbb{Z}_q$. Thus, ω satisfies the properties $\omega^N = 1 \bmod q$ and $\omega^i \neq 1 \bmod q$ for $0 < i < N$. The powers of ω are often referred to as *TFs*. The inverse NTT transformation maps $\hat{\mathbf{a}}$ to the coefficient domain $\mathbf{a} = \text{INTT}(\hat{\mathbf{a}})$ and is defined as $a_i = N^{-1} \sum_{k=0}^{N-1} \hat{a}_k \omega^{-ik} \bmod q$, where N^{-1} is the multiplicative inverse of N modulo q .

The naïve computation of NTT has a runtime of $\mathcal{O}(N^2)$. Yet, optimized algorithms, such as decimation-in-time (DIT) and decimation-in-frequency (DIF), reduce the computational complexity to $\mathcal{O}(N \log N)$ by leveraging symmetries in the TFs in a divide-and-conquer approach [18]. The DIT algorithm employs the Cooley–Tukey (CT) butterfly operation, while the DIF algorithm uses the Gentleman–Sande (GS) butterfly operation. The two butterfly configurations are illustrated in Fig. 1. Each butterfly takes two coefficients (a_i and a_j) and one TF (ω^k). The butterfly operation is the fundamental operation in the NTT transformation, as shown in Fig. 1(c) for an $N = 8$ point DIT NTT. In this example, the CT butterfly is applied iteratively across $\log_2 N = 3$ stages, with $N/2 = 4$ butterfly operations per stage.

In hardware designs, parallelism can be exploited by instantiating multiple PEs, where each PE performs a butterfly operation. The number of such PEs is denoted as n_{PE} . By increasing n_{PE} , multiple butterfly operations can be performed concurrently, lowering the latency of the transformation.

A detailed step-by-step computation of the DIT NTT using the CT butterfly is shown in Algorithm 1. The coefficients of the input polynomial $\mathbf{a}$ are assumed to be in normal order, and the output coefficients of $\hat{\mathbf{a}}$ are in bit-reversed order. In contrast, when the input polynomial is given in bit-reversed order, the output is in normal order. These two configurations are referred to as normal-to-reversed (NR) and reversed-to-normal (RN) transformations, respectively.

We also distinguish between out-of-place NTTs and in-place NTTs [12]. Out-of-place NTT designs store two polynomials on-chip. This increased memory consumption, however, allows a simpler execution flow within the transformation. In contrast, in-place NTT designs only instantiate memory for one polynomial. The resulting memory reduction has the disadvantage of a convoluted and nonuniform execution flow to avoid access conflicts to the single memory.

B. Fast Fourier Transformation

The FFT is another important building block in cryptographic designs. For example, the CKKS FHE scheme [1]

Algorithm 1 DIT NTT With CT Butterfly [19]

Input: $\mathbf{a} \in \mathcal{R}_q$ (in normal order), $\omega[N/2]$ (powers of ω)
Output: $\hat{\mathbf{a}} \leftarrow \text{NTT}(\mathbf{a}) \in \mathcal{R}_q$ (in bit-reversed order)

```

1: for ( $s = \log_2(N)$ ;  $s > 0$ ;  $s = s - 1$ ) do
2:   for ( $k = 0$ ;  $k < N/2^s$ ;  $k = k + 1$ ) do
3:      $m \leftarrow 2^s$ ;  $tw \leftarrow \omega[\text{BITREVERSE}(k) \cdot m/2]$ 
4:     for ( $j = 0$ ;  $j < m/2$ ;  $j = j + 1$ ) do
5:        $u \leftarrow \mathbf{a}[km + j]$ 
6:        $v \leftarrow \mathbf{a}[km + j + m/2] \cdot tw \bmod q$ 
7:        $\mathbf{a}[km + j] \leftarrow u + v \bmod q$ 
8:        $\mathbf{a}[km + j + m/2] \leftarrow u - v \bmod q$ 
9: return  $\mathbf{a}$ 

```

relies on FFT in the client-side operations. FFT is also part of Falcon [4], which is one post-quantum signature scheme standardized by NIST [20]. The FFT is very similar to the NTT and uses the same computation rules. However, instead of performing modular ring arithmetic, FFT relies on complex number arithmetic in $\mathbb{C}$ where each $x = x_r + ix_i \in \mathbb{C}$ is expressed as two real values $x_r, x_i \in \mathbb{R}$ with i being the imaginary unit. Hence, a hardware implementation of NTT can be extended to FFT by replacing the modular ring data path with a floating-point-based complex number datapath. Depending on the desired accuracy of the FFT, single-precision or double-precision floating-point numbers with 32- and 64-bit word sizes, respectively, can be deployed. Hence, for storing one complex number $x \in \mathbb{C}$, 64/128 bits are required for single/double precision, respectively.

The TFs ω^k in FFT are powers of the N th primitive roots of unity in $\mathbb{C}$, i.e., $\omega^k = \exp(2\pi i k/N)$. Unlike in NTT, where TFs depend on the polynomial degree N and the prime q , TFs in FFT only depend on N . In addition, TFs in FFT can be stored in ROM memory or generated on the fly.

C. Polynomial Multiplication Using NTT and FFT

NTT and FFT are commonly employed to transform data between the coefficient domain (or time domain) and the evaluation domain (or frequency domain). This is especially useful for cryptography in accelerating polynomial multiplication.

Consider two polynomials $\mathbf{a}$ and $\mathbf{b}$. Their product $\mathbf{c} = \mathbf{a} \times \mathbf{b}$ can be computed efficiently using the NTT or FFT. The process begins by applying the forward NTT/FFT to the input polynomials $\mathbf{a}$ and $\mathbf{b}$, yielding their respective evaluation representations $\hat{\mathbf{a}}$ and $\hat{\mathbf{b}}$. In the evaluation domain, the multiplication is simplified to a coefficient-wise operation, expressed as $\hat{\mathbf{c}} = \hat{\mathbf{a}} \odot \hat{\mathbf{b}}$. This operation is followed by an inverse NTT/FFT to transform $\hat{\mathbf{c}}$ back to the coefficient domain $\mathbf{c}$.

Note that using NTT or FFT for polynomial multiplication inherently increases the degree of the resulting polynomial $\mathbf{c}$, which must be managed efficiently. Two common techniques are used to address this challenge. The first is zero-padding, which extends the input polynomials with zeros to double their length, necessitating a $2N$ -point transformation. While effective, this approach increases computational and memory requirements. The second technique, known as NWC [7], eliminates the need for zero-padding by leveraging $2N$ th roots of unity denoted as ψ . Algorithm 2 shows the NWC-based polynomial multiplication involving pre and postprocessing

Algorithm 2 NWC-Based Polynomial Multiplication [11]

Input: $\mathbf{a}, \mathbf{b} \in \mathcal{R}_q$, ψ : primitive $2N$ -th root of unity in $\mathbb{Z}_q$
Output: $\mathbf{c} \leftarrow \mathbf{a} \times \mathbf{b} \in \mathcal{R}_q$

```

1:  $\hat{\mathbf{a}} \leftarrow \text{NTT}(\mathbf{a} \odot (1, \psi^1, \dots, \psi^{N-1})) = \text{MNTT}(\mathbf{a})$ 
2:  $\hat{\mathbf{b}} \leftarrow \text{NTT}(\mathbf{b} \odot (1, \psi^1, \dots, \psi^{N-1})) = \text{MNTT}(\mathbf{b})$ 
3:  $\hat{\mathbf{c}} \leftarrow \text{INTT}(\hat{\mathbf{a}} \odot \hat{\mathbf{b}}) \odot (1, \psi^{-1}, \dots, \psi^{-(N-1)}) = \text{MINTT}(\hat{\mathbf{a}} \odot \hat{\mathbf{b}})$ 
4: return  $\mathbf{c}$ 

```

by coefficient-wise multiplications with ψ^k . The pre and postprocessing steps can be merged into the NTT/INTT transformation [21]. We refer to this NTT type as *merged* NTT, denoted as MNTT/MINTT. Merged NTT significantly reduces computational and memory overhead but introduces higher complexity [18]. In addition, the TFG (explained in Section II-D) is more complicated in MNTT due to the convoluted TF sequence. While Algorithm 2 shows NWC-based NTT, NWC can be directly applied to FFT as well.

D. Twiddle Factors in NTT and FFT

The TFs ω^k , which are powers of the root of unity in their domain, are critical in NTT and FFT computations. Their management directly influences hardware scalability and performance. Two approaches for TF handling are commonly employed: 1) precomputing and storing TFs, or 2) TFG. In TFG, the TF are dynamically computed during execution without storing them.

1) *Stored Twiddle Factors*: The stored approach precomputes and saves all required TFs in memory, making it simple and efficient for small N . However, this method scales poorly for large polynomials since memory requirements grow linearly with N . Moreover, FHE schemes like CKKS use multiple moduli $q_0 \dots q_{L-1}$, thereby storing multiple sets of TFs—one for each modulus.

2) *On-the-Fly Generated Twiddle Factors*: Dynamically computing TFs during execution reduces memory usage. This approach is well-suited for systems with large N or frequent modulus switching, as it removes the need to store multiple sets of precomputed factors. Although this approach introduces computational overhead, it lowers memory consumption. In hardware designs, only a few base TFs must be stored to fill the generation pipeline initially. Yet, this requires a dedicated processing sequence as TFs cannot be generated efficiently in arbitrary order. This especially applies to merged NTT designs.

III. OPENNTT'S TWIDDLE FACTOR GENERATION

It is challenging to ensure an efficient TFG for generic parameter sets. This particularly applies to both algorithmic and hardware design perspectives. First, from the algorithmic perspective, conflict-free memory accesses must be combined with linear TFG to ensure a fully pipelined design. Second, from the hardware design perspective, different synthesis-time options are desirable to allow customization of the hardware accelerator for different key benchmarks (low area, high performance, etc.).

This section presents both our proposed processing order supporting efficient TFG and our synthesis-time optimization

strategies. Our optimizations allow for low memory consumption (MemOpt strategy) or higher design frequencies through improved routing capabilities (RoutOpt strategy).

A. Processing Order During Transformation

A fully pipelined hardware design for NTT or FFT must ensure conflict-free access to on-chip memory, as inefficiencies would arise due to stalls in the computational logic. Yet, the requirement of conflict-free memory accesses during in-place transformations introduces certain limitations to the processing order of the transformation: The required address sequence for reading and writing to on-chip memory is dependent on the coefficient ordering (NR and RN) and even changes across consecutive stages [22]. In addition, the TFs can not be generated efficiently in an arbitrary order. Therefore, additional limitations need to be considered for the possible execution flow in NTT and FFT transformations.

In [16], a generic address generation algorithm is derived for NR NTT. This address generation is presented in Algorithm 3. We extend Algorithm 3 for NR transformations and derive the TFG-friendly address generation for RN transformations. Thereby, we follow a similar approach as in [16]. The resulting algorithm for RN transformations is shown in Algorithm 4. We observe from Algorithm 4 that in stages where $s < \log_2(N) - 1 - \log_2(n_{PE})$, the RN transformation cannot use a linear address sequence. Instead, the address sequence is computed as presented in lines 7–12 in Algorithm 4. This computation first splits the index i into a high part h and a low part l , depending on the current stage s . Thereafter, the low part is rotated left and bit-reversed. Finally, the h is appended to l (l is in the most significant bits), yielding the desired address. Yet, in the last $\log_2(n_{PE})$ stages, any address sequence in RN transformation leads to nonconflicting memory accesses. Hence, we apply a linear address sequence in this case for RN transformations. This contrasts with the NR transformation in Algorithm 3 where the *first* $\log_2(n_{PE})$ stages follow a linear processing order.

The presented Algorithms 3 and 4 compute the desired address sequence generically for all N and n_{PE} . Following this sequence, we present our two optimization strategies for TFG in the next sections.

B. Routing Optimized TFG Configuration

The TFG in OpenNTT should support a wide range of parameters, including polynomial degree N or transformation type (NR or RN). Moreover, OpenNTT is flexible in the number of PEs (n_{PE}) instantiated, allowing a tradeoff between performance and area consumption.

During the transformation, each of the n_{PE} PEs requires a unique TF sequence that depends on the parameters. Hence, we instantiate one dedicated multiplier unit (TwGen module) for each PE to compute its TF sequence. The multiplier is fully pipelined and has a latency of d_{Mul} stages, where d_{Mul} depends on the prime size or the floating point precision, respectively. Before each stage of a transformation, the multiplier pipeline must be filled with proper TFs. These TFs are obtained from a small ROM memory (TwROM) containing the corresponding

Algorithm 3 Processing Order for NR Transformation [16]

Input: N, n_{PE}
Output: $addr[N/2/n_{PE} \cdot \log_2(N)]$: array with the address sequence

```

1:  $addr \leftarrow []$ 
2: for  $s = 0$  to  $\log_2(N) - 1$  do           ▷ stage iterator  $s$ 
3:   for  $i = 0$  to  $N/2/n_{PE}$  do
4:     if  $s < \log_2(n_{PE})$  then
5:        $addr.APPEND(i)$                      ▷ linear order
6:     else
7:        $b \leftarrow \log_2(N/2/n_{PE})$            ▷ bit-width of address
8:        $l\_bits \leftarrow b - s + \log_2(n_{PE})$    ▷ bit-width of low part
9:        $h \leftarrow i[b-1:l\_bits]$ 
10:       $l \leftarrow i[l\_bits-1:0]$ 
11:       $h \leftarrow BITREVERSE(h)$              ▷ bit-reverse high part
12:       $l \leftarrow l \ggg 1$                    ▷ rotate low part
13:       $addr.APPEND(\{h, l\})$                  ▷ bit-wise append  $h$  and  $l$ 
14: return  $addr$ 

```

Algorithm 4 Processing Order for RN Transformation

Input: N, n_{PE}
Output: $addr[N/2/n_{PE} \cdot \log_2(N)]$: array with the address sequence

```

1:  $addr \leftarrow []$ 
2: for  $s = 0$  to  $\log_2(N) - 1$  do           ▷ stage iterator  $s$ 
3:   for  $i = 0$  to  $N/2/n_{PE}$  do
4:     if  $s \geq \log_2(N) - 1 - \log_2(n_{PE})$  then
5:        $addr.APPEND(i)$                      ▷ linear order
6:     else
7:        $b \leftarrow \log_2(N/2/n_{PE})$            ▷ bit-width of address
8:        $h \leftarrow i[b-1:b-s]$ 
9:        $l \leftarrow i[b-s-1:0]$ 
10:       $l \leftarrow l \lll 1$                    ▷ rotate low part
11:       $l \leftarrow BITREVERSE(l)$              ▷ bit-reverse low part
12:       $addr.APPEND(\{l, h\})$                  ▷ bit-wise append  $l$  and  $h$ 
13: return  $addr$ 

```

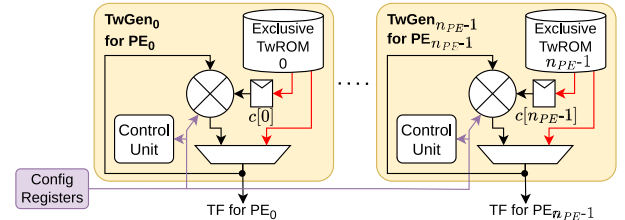


Fig. 2. Architecture of our RoutOpt TwGen module.

subset of TFs. Based on this initial filling, the remaining TFs are computed by repeated multiplications.

In our RoutOpt TFG, each PE has an exclusive TwROM in its TwGen module, as shown in Fig. 2. This allows the placement of these hardware resources near the PE, leading to improved routability and shorter critical paths. Thus, higher frequencies can be supported. The TF ROMs—instantiated once per PE—contain the minimal set of initial TFs needed by the supplied PE. This initial set is used to fill the multiplier pipeline before each transformation stage. Note that some TFs are stored redundantly in multiple TwROM instances due to missing interconnects between different PEs.

Table I shows an example of two stages within a DIF NR transformation for $N = 128$, $n_{PE} = 4$. The table indicates the power of TFs (ω^i) needed by each PE and each transformation step. The TFs forwarded to the PE are in bold characters,

TABLE I
ROUTING OPTIMIZED TFG FOR DIF NR

Stage 0				
Initialize registers: $c \leftarrow [3, 3, 3, 3]$ from TwROM				
Step	TF for PE ₀	TF for PE ₁	TF for PE ₂	TF for PE ₃
0	0 = 0	16 = 16	32 = 32	48 = 48
1	1 = 1	17 = 17	33 = 33	49 = 49
2	2 = 2	18 = 18	34 = 34	50 = 50
3	3 = 0 + $c[0]$	19 = 16 + $c[1]$	35 = 32 + $c[2]$	51 = 48 + $c[3]$
4	4 = 1 + $c[0]$	20 = 17 + $c[1]$	36 = 33 + $c[2]$	52 = 49 + $c[3]$
5	5 = 2 + $c[0]$	21 = 18 + $c[1]$	37 = 34 + $c[2]$	53 = 50 + $c[3]$
⋮	⋮	⋮	⋮	⋮
15	15 = 12 + $c[0]$	31 = 28 + $c[1]$	47 = 44 + $c[2]$	63 = 60 + $c[3]$
Stage 1				
Initialize registers: $c \leftarrow [6, 6, 6, 6]$ from TwROM				
Step	TF for PE ₀	TF for PE ₁	TF for PE ₂	TF for PE ₃
0	0 = 0	32 = 32	0 = 0	32 = 32
1	2 = 2	34 = 34	2 = 2	34 = 34
2	4 = 4	36 = 36	4 = 4	36 = 36
3	6 = 0 + $c[0]$	38 = 32 + $c[1]$	6 = 0 + $c[2]$	38 = 32 + $c[3]$
4	8 = 2 + $c[0]$	40 = 34 + $c[1]$	8 = 2 + $c[2]$	40 = 34 + $c[3]$
5	10 = 4 + $c[0]$	42 = 36 + $c[1]$	10 = 4 + $c[2]$	42 = 36 + $c[3]$
⋮	⋮	⋮	⋮	⋮
15	30 = 24 + $c[0]$	62 = 56 + $c[1]$	30 = 24 + $c[2]$	62 = 56 + $c[3]$

Notation is in $\log_{\omega}$: 3 = 1 + 2 refers to $\omega^3 = \omega^1 \cdot \omega^2$.

while the computation rule for these TFs is in normal font. In addition, Algorithm 5 details computation steps for the TFs in RoutOpt.

Before the transformation starts, each TwGen_i loads a TF $c[i]$ from TwROM into a local register, as shown in line 2 in Algorithm 5. Then, considering a multiplier latency of $d_{\text{Mul}} = 3$ clock cycles (cc), the first three TF of each stage are obtained from the TwROM (marked in red in Table I and shown in lines 4–7 in Algorithm 5). In TwGen_0 , stage 0, this would be ω^0 , ω^1 , and ω^2 . The three TF from TwROM are directly forwarded to the PE performing the butterfly operation. Simultaneously, they are provided as input to the TwGen's multiplier to fill the multiplier's pipeline. During these multiplications, the preloaded TF $c[i]$ (e.g., $c[i] = \omega^3$ in stage 0) serves as multiplicand for generating the TF sequence from step 3 onward, which avoids TwROM access for the remaining stage. Thereby, only $c[i]$ is kept in a register. After the multiplier pipeline was filled with d_{Mul} TF from TwROM (lines 4–7 in Algorithm 5), the multiplier outputs one computed TF per cc (starting in line 8) for the rest of the stage. The output TF is directly fed back and serves as input to another multiplication by $c[i]$ utilizing the TwGen multiplier (lines 9–11). This allows a streamlined computation of the TF sequence.

After a certain stage s is completed, we use a very similar approach for TFG in stage $s + 1$. The registers $c[i]$ are initialized with new values from TwROM, and the multiplier pipeline is again filled with TFs obtained from TwROM as indicated in Table I. To avoid pipeline stalls during stage transition, loading $c[i]$ for stage $s + 1$ is performed during the last steps of stage s (lines 12–13 in Algorithm 5). This strategy allows a fully pipelined TFG without stall cycles.

Fig. 2 shows the hardware architecture of our TwGen module following the RoutOpt strategy. The red arrows show data flow during the stage initialization to fill the multiplier pipeline from TwROM. In addition, the constant multiplicands $c[i]$ are also obtained from TwROM. Once the multiplier pipeline is filled, the generated TFs are fed back along the

Algorithm 5 TF Generation for PE_i in DIF NR RoutOpt

Input: TwROM[i]: TwROM memory content for PE_i ($0 \leq i < n_{\text{PE}}$),

Input: n_{PE} : number of PEs, d_{Mul} : multiplier latency

Output: TF $_i$: Twiddle factor sequence for PE_i

```

1: TF $_i \leftarrow []$ 
2:  $c[i] \leftarrow \text{TwROM}[i].\text{GETDATA}()$   $\triangleright$  Initialization for stage 0
3: for  $s = 0$  to  $\log_2(N) - 1$  do  $\triangleright$  Stage iterator
4:   repeat  $d_{\text{Mul}}$  times  $\triangleright$  Steps 0 to 2 in Table I
5:      $\omega \leftarrow \text{TwROM}[i].\text{GETDATA}()$ 
6:     TF $_i.\text{APPEND}(\omega)$ 
7:     Start multiplication of  $\omega \cdot c[i]$   $\triangleright$  Result ready after  $d_{\text{Mul}}$  cc
   /* From now on, the multiplier outputs one result in each cc */
8:   repeat  $\frac{N}{2n_{\text{PE}}} - 2 \cdot d_{\text{Mul}}$  times  $\triangleright$  Step 3 onward in Table I
9:      $\omega \leftarrow$  multiplier output
10:    TF $_i.\text{APPEND}(\omega)$ 
11:    Start multiplication of  $\omega \cdot c[i]$   $\triangleright$  Result ready after  $d_{\text{Mul}}$  cc
12:    if  $s < \log_2(N) - 1$  then  $\triangleright$  Initialize for next stage
13:       $c[i] \leftarrow \text{TwROM}[i].\text{GETDATA}()$ 
14:    repeat  $d_{\text{Mul}}$  times  $\triangleright$  No multiplications issued
15:       $\omega \leftarrow$  multiplier output
16:      TF $_i.\text{APPEND}(\omega)$ 
17: return TF $_i$ 

```

black arrows until the stage is done. Thereafter, the TwGen module is prepared for the next stage by again retrieving TFs from TwROM. This methodology directly applies to the remaining stages of the DIF NR transformation. Moreover, similar techniques can easily be extended to other transformation types, such as DIT and RN transformations.

C. Memory Optimized TFG Configuration

In contrast to RoutOpt, which optimizes for high frequency, our MemOpt configuration for TFG aims to minimize the number of TFs stored in ROM. This contrasts with RoutOpt, where a higher number of TFs are needed in memory. Table II illustrates our corresponding MemOpt optimizations and shows the TF computation within the first two stages of a DIF NR transformation. The chosen parameters for this example are $N = 128$, $n_{\text{PE}} = 4$, and $d_{\text{Mul}} = 4$ cycles. Yet, our MemOpt methodology is not restricted to these parameters. In addition, Algorithm 6 gives detailed steps for our MemOpt TFG, and Fig. 3 shows the hardware architecture.

The execution in our MemOpt design starts with an initialization phase (lines 2–6 in Algorithm 6). First, the initialization sequentially loads $1 + n_{\text{PE}}$ TFs from the shared TwROM and stores them in the registers c and $t[i]$ within the TwGen_i modules ($0 \leq i < n_{\text{PE}}$). The corresponding data path is shown in red in Fig. 3. In addition, the initialization is indicated in the top line of Table II with concrete values for the demonstration parameters. The second part of initialization fills the multiplier pipelines (lines 4–6 in Algorithm 6) with d_{Mul} TFs obtained from TwROM. Note that all TwGen modules receive the same TF in this step, and TwGen_i issues a multiplication of the obtained TF by $t[i]$. In the example of Table II, the $d_{\text{Mul}} = 4$ TF from TwROM (ω^0 to ω^3) are highlighted in red. In total, the initialization loads $1 + n_{\text{PE}} + d_{\text{Mul}}$ many TFs from TwROM.

Upon completion of the initialization, the multiplier pipelines in TwGen modules are fully filled, and the actual transformation starts. The first set of produced TFs (step 0 in stage 0 in Table II)

TABLE II
MEMORY OPTIMIZED TFG FOR DIF NR [16]

Stage 0				
Initialize registers: $c \leftarrow 4$ from TwROM, $t \leftarrow [0, 16, 32, 48]$ from TwROM				
Step	TF for PE ₀	TF for PE ₁	TF for PE ₂	TF for PE ₃
0	$0 = 0 + t[0]^{\textcircled{1}\textcircled{2}}$	$16 = 0 + t[1]$	$32 = 0 + t[2]^{\textcircled{1}}$	$48 = 0 + t[3]$
1	$1 = 1 + t[0]$	$17 = 1 + t[1]$	$33 = 1 + t[2]$	$49 = 1 + t[3]$
2	$2 = 2 + t[0]^{\textcircled{2}}$	$18 = 2 + t[1]$	$34 = 2 + t[2]$	$50 = 2 + t[3]$
3	$3 = 3 + t[0]$	$19 = 3 + t[1]$	$35 = 3 + t[2]$	$51 = 3 + t[3]$
4	$4 = 0 + c^{\textcircled{2}}$	$20 = 16 + c$	$36 = 32 + c$	$52 = 48 + c$
5	$5 = 1 + c$	$21 = 17 + c$	$37 = 33 + c$	$53 = 49 + c$
6	$6 = 2 + c^{\textcircled{2}}$	$22 = 18 + c$	$38 = 34 + c$	$54 = 50 + c$
7	$7 = 3 + c$	$23 = 19 + c$	$39 = 35 + c$	$55 = 51 + c$
8	$8 = 4 + c^{\textcircled{3}}$	$24 = 20 + c$	$38 = 36 + c$	$56 = 52 + c$
⋮	⋮	⋮	⋮	⋮
15	$15 = 11 + c$	$31 = 27 + c$	$47 = 43 + c$	$63 = 59 + c$
Stage 1				
Setup registers: $c \leftarrow 8$ from $\textcircled{3}$ in Stage 0, $t \leftarrow [0, 32, 0, 32]$ from $\textcircled{1}$ in Stage 0, FIFO $\leftarrow [0, 2, 4, 6]$ from $\textcircled{2}$ in Stage 0				
Step	TF for PE ₀	TF for PE ₁	TF for PE ₂	TF for PE ₃
0	$0 = 0 + t[0]^{\textcircled{1}\textcircled{2}}$	$32 = 0 + t[1]$	$0 = 0 + t[2]^{\textcircled{1}}$	$32 = 0 + t[3]$
1	$2 = 2 + t[0]$	$34 = 2 + t[1]$	$2 = 2 + t[2]$	$34 = 2 + t[3]$
2	$4 = 4 + t[0]^{\textcircled{2}}$	$36 = 4 + t[1]$	$4 = 4 + t[2]$	$36 = 4 + t[3]$
3	$6 = 6 + t[0]$	$38 = 6 + t[1]$	$6 = 6 + t[2]$	$38 = 6 + t[3]$
4	$8 = 0 + c^{\textcircled{2}}$	$40 = 32 + c$	$8 = 0 + c$	$40 = 32 + c$
5	$10 = 2 + c$	$42 = 34 + c$	$10 = 2 + c$	$42 = 34 + c$
6	$12 = 4 + c^{\textcircled{2}}$	$44 = 36 + c$	$12 = 4 + c$	$44 = 36 + c$
⋮	⋮	⋮	⋮	⋮
15	$30 = 22 + c$	$62 = 54 + c$	$30 = 22 + c$	$62 = 54 + c$

Notation is in $\log_{\omega}$: $3 = 1 + 2$ refers to $\omega^3 = \omega^1 \cdot \omega^2$.

is ω^0 , ω^{16} , ω^{32} , and ω^{48} . Each TwGen module forwards its generated TF to the PE, which performs the butterfly operation. Simultaneously, the TwGen module feeds the TF back and starts a multiplication with c stored in a register (lines 9–11 in Algorithm 6). This computes the remaining TF sequence for stage 0, starting from step 4 in Table II. The corresponding datapath is colored in black in Fig. 3.

While the values for the registers c and $t[i]$ in stage $s = 0$ stem from the TwROM, their values for the following stages are obtained from the produced TFs in the prior stage to minimize TwROM consumption. In particular, we collect the values for c and $t[i]$ for stage $s > 0$ from stage $s - 1$ using a generic and scalable method presented in lines 12–15 in Algorithm 6. In Table II, the collected TFs for c and $t[i]$ are indicated with $\textcircled{3}$ and $\textcircled{1}$, respectively. In addition to c and $t[i]$, d_{Mul} many TFs are needed to fill the multiplier pipeline at the beginning of stage s . These TFs are collected during stage $s - 1$ and fed into a FIFO with d_{Mul} elements (blue datapath in Fig. 3). The FIFO is filled with the TF produced by TwGen₀ in even steps, as shown in lines 16 and 17 in Algorithm 6 and marked with $\textcircled{2}$ in Table II. In the transition from stage $s - 1$ to stage s , the multipliers in TwGen are provided with the FIFO content during the last d_{Mul} steps of stage $s - 1$ (indicated in lines 22–24 in Algorithm 6 and colored blue in Table II). The multipliers then output the desired TFs after d_{Mul} cc, which is just in time for the beginning of stage s , thereby avoiding pipeline stalls.

The remaining stages follow this procedure to generate the TFs on the fly. We also extend this strategy to other transformation types and parameter sets in our open-source repository. Note that the shared TwROM in our MemOpt architecture is only required during the initialization phase (i.e., only the red TFs in Table II are from TwROM). Hence, the TwROM

Algorithm 6 TF Generation for PE_i in DIF NR MemOpt

Input: TwROM: TwROM memory content (shared among all PEs)

Input: d_{Mul} : multiplier latency, n_{PE} : number of PEs

Output: TF_i: Twiddle factor sequence for PE_i

```

1: TFi  $\leftarrow []$ 
   /* Initialization for stage  $s = 0$  */
2:  $c \leftarrow \text{TwROM.GETDATA}()$ 
3:  $t[i] \leftarrow \text{TwROM.GETDATA}()$   $\triangleright$  Done serially for all PE
4: repeat  $d_{Mul}$  times  $\triangleright$  Issue first  $d_{Mul}$  multplies.
5:    $\omega \leftarrow \text{TwROM.GETDATA}()$   $\triangleright$  All PE get the same  $\omega$ 
6:   Start multiplication of  $\omega \cdot t[i]$   $\triangleright$  Result ready after  $d_{Mul}$  cc
   /* From now on, the multiplier outputs one result each cc and
   the actual transform starts. TwROM no longer needed */
7: for  $s = 0$  to  $\log_2(N) - 1$  do  $\triangleright$  Stage iterator
8:   for  $j = 0$  to  $\frac{N}{2n_{PE}} - d_{Mul} - 1$  do
9:      $\omega \leftarrow$  multiplier output
10:    TFi.APPEND( $\omega$ )
11:    Start multiplication of  $\omega \cdot c$   $\triangleright$  Result ready after  $d_{Mul}$  cc
12:    if  $i = 0$  and  $j = N/4/n_{PE}$  then
13:       $c_n \leftarrow \omega$   $\triangleright c_n$  is  $c$  in next stage
14:      if  $j = 0$  and  $i$  is even then  $\triangleright$  Setup  $t[i]$  for next stage
15:         $t[i/2] \leftarrow \omega$ ,  $t[(i + n_{PE})/2] \leftarrow \omega$ 
16:      if  $i = 0$  and  $j < 2d_{Mul}$  and  $j$  is even then
17:        FIFO.PUTDATA( $\omega$ )  $\triangleright$  Fill FIFO for next stage
   /* Last steps of stage  $s$  and transition to stage  $s + 1$  */
18:    $c \leftarrow c_n$   $\triangleright$  Setup  $c$  for next stage
19:   for  $j = \frac{N}{2n_{PE}} - d_{Mul}$  to  $\frac{N}{2n_{PE}} - 1$  do
20:      $\omega \leftarrow$  multiplier output
21:     TFi.APPEND( $\omega$ )
22:     if  $s < \log_2(N) - 1$  then  $\triangleright$  Fill pipeline for next stage
23:        $\omega_n \leftarrow \text{FIFO.GETDATA}()$ 
24:       Start multiplication of  $\omega_n \cdot t[i]$ 
25: return TFi

```

only contains a small set of TFs, minimizing the memory requirements. While this effectively lowers the memory consumption, the critical path indicated in Fig. 3 reduces the achievable clock frequency compared to RoutOpt. Thus, our OpenNTT tool provides design-time tradeoffs between latency and ROM consumption of the produced hardware architectures. Section V presents a detailed comparison of MemOpt and RoutOpt.

D. Multiplier Architecture and Parameter Flexibility of TwGen

Our RoutOpt and MemOpt designs allow a high degree of parameter flexibility through synthesis-time and run-time configuration. We refer to Section IV-A for an extensive explanation.

The synthesis-time configuration fixes the polynomial degree N , n_{PE} , and the transformation type. It also instantiates and configures the TwGen and PE modules. The run-time configurability of OpenNTT allows selecting between forward or inverse transformations and the NTT modulus used. Therefore, the host system writes to configuration registers shown in Figs. 2 and 3, which select the transformation direction and the NTT modulus. To change the NTT modulus at runtime, a word-level Montgomery modular reduction for generic NTT-friendly primes is used as in [11]. Moreover, our design allows variable-width moduli by instantiating integer multiplier and Montgomery reduction according to the *largest* modulus. This

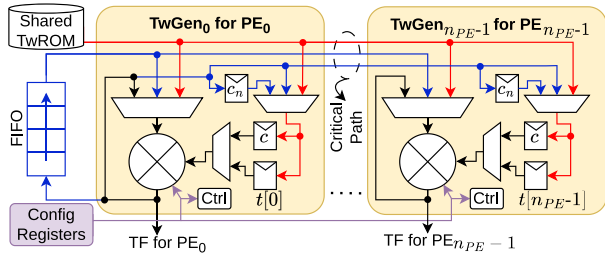


Fig. 3. Architecture of MemOpt TFG.

readily supports smaller moduli, which is required in RNS-based FHE applications [23]. Next to the multipliers in TwGen, the TwROM content also reflects the selected parameter set. Specifically, the TwROM contains one dedicated set of initial TFs, either following MemOpt or RoutOpt, for each modulus. For example, consider the MemOpt DIF NR case in Table II. Therein, a set of nine TFs is stored in TwROM per modulus. Thus, the TFs for the first modulus are stored in TwROM locations 0–8 (base address: 0). The TFs for the second modulus are in locations 9–17 (base address: 9), etc. The base address is provided by configuration registers at runtime to select the TF set. The control unit in TwGen adds the offset to select a concrete TF. We use the same strategy in our RoutOpt designs.

IV. OVERALL DESIGN OF OPENNTT

This section details the parameter flexibility of OpenNTT and shows our fully parametrized hardware architecture. Finally, we explain the OpenNTT hardware design flow.

A. Supported Parameters in OpenNTT

Our OpenNTT tool generates efficient FFT and NTT hardware accelerators for various scenarios. Therefore, OpenNTT shows flexibility in scheme-specific parameters, such as polynomial degree N , coefficient types (modular ring and floating-point), and hardware implementation-specific considerations. The latter includes our MemOpt and RoutOpt design strategies to balance performance and memory consumption. Moreover, the number of PEs is adaptable to different resource constraints.

Table III gives an overview of the design parameters and their supported values. OpenNTT can compile standalone forward and inverse FFT/NTT architectures, wherein only one transformation direction is supported. This saves hardware resources compared to unified architectures (Unif.), which can perform both forward and inverse FFT/NTT transformations. To select between FFT and NTT hardware architectures, users provide a flag to the toolchain, which subsequently instantiates either the modular ring datapath or the complex number datapath. The complex number datapath complies with the IEEE754 floating-point standard, which allows a straightforward adoption of OpenNTT's FFT designs.

Other important configurations supported by OpenNTT include the polynomial degree N and the number of PEs n_{PE} , as listed in Table III. For NTT designs, the desired primes q and the roots of unity ω can be specified. Alternatively, the

TABLE III
PARAMETER SPACE OF OPENNTT

Design Parameter	Supported Values
Standalone forward types	DIT_NR, DIF_NR, M_DIT_NR
Standalone inverse types	DIT_RN, DIF_RN, M_DIF_RN
Unified types (Unif.)	DIT_NR-DIT_RN, M_DIT_NR-M_DIF_RN
Transformation type	FFT, NTT
Optimization strategy	MemOpt, RoutOpt
N	Powers of two, at least 128
n_{PE}	Powers of two (Limited by FPGA resources)
Fixed q	True, False
List of q or $\log_2(q)$	at least 1 element
List of ω	Twiddle factors for each q or auto-select ω
Multiplier latency d_{Mul}	≤ 32 (Easily extendable to higher latencies)
Coefficient arithmetic	No, Mul_Add_Sub
n_{Poly} : Nr polys in ROM	Any (Limited by FPGA resources)

OpenNTT tool can overtake the prime and TFG according to the specified prime-size $\log_2(q)$. Furthermore, OpenNTT supports multiple primes $q_0 \dots q_{L-1}$ efficiently in one architecture. This capability is especially useful for FHE accelerators where frequent prime changes occur due to residue-number system splitting [24]. A seamless transition between primes can be performed without off-chip communication due to the on-the-fly TFG.

OpenNTT also provides optional coefficient arithmetic support. Thereby, the tool integrates the functionality of performing coefficient-wise addition, subtraction, and multiplication into the hardware unit. The coefficient-wise operations reuse the existing data path within the PEs to perform arithmetic operations. Moreover, a configurable number of polynomials n_{Poly} can be stored in the hardware accelerator. These are addressed via instruction operands and serve as input operands for coefficient-wise operations and transformations.

B. Overall Hardware Architecture

OpenNTT incorporates a fully parametrizable hardware architecture shown in Fig. 4. The architecture is carefully designed to reach high clock frequencies, thereby allowing performant FFT and NTT hardware accelerators. Moreover, it supports a wide range of parameter sets relevant to PQC and FHE applications, as shown in Table III.

OpenNTT's hardware architecture instantiates n_{PE} many PEs (PE_0 to $PE_{n_{PE}-1}$), as shown in Fig. 4. Each PE contains one butterfly circuit and either an exclusive or external TFG module (TwGen), depending on the used strategy. The TwGen internally uses a dedicated multiplier to compute the TF sequence for the PE as described in Section III. Furthermore, the TwGen modules either follow our RoutOpt or our MemOpt option to store precomputed TFs. In RoutOpt, each TwGen module has an exclusive TF ROM, colored green in Fig. 4. This avoids data exchange across PEs, leading to better routability in dense designs. Contrarily, in the MemOpt strategy, the shared TF ROM is used (indicated in blue). The single shared ROM distributes TFs to all PEs along the blue datapath in Fig. 4 to initially fill the pipeline. This reduces the memory demand compared to RoutOpt since TFs are not redundantly stored (see Section III-C).

In addition to the TF, each PE consumes two input coefficients and produces two output coefficients per cc. A

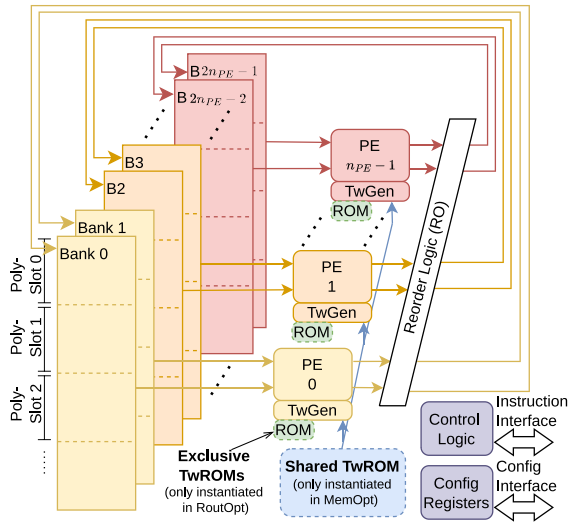


Fig. 4. Configurable hardware architecture of OpenNTT.

total of two memory banks ($2n_{PE}$) are required per PE to provide the necessary memory bandwidth. These banks form the memory subsystem in OpenNTT, as shown on the left in Fig. 4. The memory subsystem can hold a user-defined number of polynomials, n_{Poly} , where each polynomial is stored in one PolySlot spanning over all banks. This layout is indicated in Fig. 4 as PolySlot 0, PolySlot 1, and PolySlot 2. It allows the processing of multiple polynomials, e.g., during coefficient-wise multiplication. OpenNTT uses dedicated control signals to select the targeted PolySlot for each operation.

The control logic, shown bottom right in Fig. 4, controls the overall operation. Depending on the configuration (given by the configuration registers), the control logic accepts instructions for forward and inverse transformation as well as coefficient-wise addition, subtraction, and multiplication. The control logic generates the addresses for the memory subsystem according to the conflict-free memory access pattern and the efficient TFG order as shown in Tables I and II. Subsequently, the ReorderLogic is responsible for storing the PE's results in proper memory locations to ensure a stall-free NTT and FFT computation.

The presented hardware architecture is implemented in SystemVerilog and fully parametrized. The parameterization allows for an adaptation to various parameter sets for FFT and NTT with minimal user intervention.

C. OpenNTT Design Flow

The fully flexible design flow of OpenNTT is illustrated in Fig. 5. The flow covers all stages of NTT and FFT hardware generation based on the input parameter set (left in Fig. 5). Using this parameter set, OpenNTT executes a Python script to generate parameterized SystemVerilog code. In particular, the hardware design (*.sv files in Fig. 5) is fully parametrized, and the Python script generates a configuration file (*.svh) which assigns concrete values to the *.sv parameters in the top module instance. In addition, OpenNTT provides testbenches (*.tb) and test vectors (*.tv) to validate the functionality of the compiled design via Vivado's behavioral simulation.

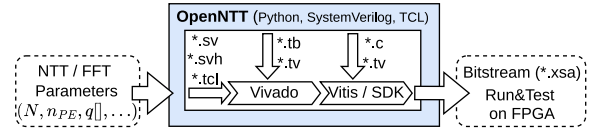


Fig. 5. Illustration of OpenNTT's hardware design flow.

In the next step, OpenNTT uses a template RTL wrapper with a CPU IP (e.g., Xilinx' Microblaze [25] or ARM core IPs [26]) to interface with the OpenNTT hardware modules. This interface uses memory-mapped IO for the configuration registers and a DMA to stream the large polynomial data. In addition, OpenNTT prepares C code (*.c in Fig. 5), which is executed on the CPU and operates the implemented hardware design on real FPGAs. Finally, the functionality on the FPGA is verified using prepared test vectors (*.tv). OpenNTT automates all these steps, to lower the effort for the user.

V. RESULTS OF MEMOPT AND ROUTOPT DESIGNS

This section presents results and compares OpenNTT's MemOpt and RoutOpt architectures. All results are obtained from implementation runs on Vivado 2022.2 with default settings. The correctness of each design is validated on real FPGAs.

A. Resource Utilization

The memory involved in NTT and FFT designs is a major factor in the overall resource utilization. Hence, reducing the ROM memory for TFs is an important design strategy. To show the effectiveness of our MemOpt and RoutOpt strategies, we consider FHE-typical NTT parameters [23], [27]: We assume eight different primes q_0 to q_7 each having $\log_2(q_i) = 54$ bits and $n_{PE} = 4$.

Table IV shows the number of TFs needed in TwROM memory across different N for 1) stored TFs; 2) generated TFs using our RoutOpt; and 3) generated TFs using our MemOpt strategy. For stored TFs, we observe a linear dependency between N and the memory consumption, reaching up to 65.5-k TFs in memory for $N = 2^{14}$. In contrast, generated TFs just show a slight memory increase due to the lower number of TFs kept in TwROM. For example, in FHE-typical $N = 2^{14}$ -point NTTs, TFG lowers the number of TFs by one to two orders of magnitude compared to stored TFs. Table IV also reports the BRAM consumption. Therein, RoutOpt uses four BRAMs to store the required TFs while MemOpt only requires one BRAM. Regarding stored TFs, our MemOpt and RoutOpt reduce the BRAM count by $3\times$ to $96\times$.

Comparing our RoutOpt and MemOpt strategies, we require up to $20\times$ more TFs in RoutOpt than in MemOpt. For example, for $N = 2^{11}$, RoutOpt stores 1.7-k TFs while MemOpt only stores 88 TFs. This is caused by the redundant storing of TFs within TwROM instances of different PEs, as discussed in Section III-B. Nevertheless, the number of TFs in RoutOpt is one order of magnitude less than in stored TFs and does not grow significantly with larger N . The key benefit of our RoutOpt strategy is the improved routability, which allows higher frequencies and hence higher performance.

TABLE IV
COMPARISON OF TwROM CONSUMPTION FOR MEMOPT AND ROUTOPT

N	Nr Twiddle Factors in TwROM			BRAMs for TwROM			
	Stored	RoutOpt	MemOpt	Stored	RoutOpt	MemOpt	
2^{11}	8,192	1,728	88	12	4	1	
2^{12}	16,384	1,792	96	24	4	1	
2^{13}	32,768	1,728	104	48	4	1	
2^{14}	65,536	1,792	112	96	4	1	

B. Achieved Frequency and Latency in Our Designs

As discussed in the previous section, our RoutOpt TFG has a higher memory usage than MemOpt due to the redundant TFs. Yet, the RoutOpt approach increases the achievable clock frequency in dense designs. This is highlighted in Table V, which reports the maximum frequencies for different parameter sets. We observe that our RoutOpt designs achieve up to 51-MHz higher clock frequencies than our MemOpt due to the independent TwGen modules. Unlike the frequency, the cycle counts in MemOpt and RoutOpt designs are nearly identical as reported in Table V. MemOpt needs <1% more cc caused by its deeper pipeline taking longer for initial filling. Therefore, the up to 12% lower latency in μs of RoutOpt mainly stems from its higher frequency, whereas the cycle count contributes negligibly to the speedup.

C. Power and Energy Consumption in Our Designs

Table V reports the power and energy consumption of RoutOpt and MemOpt designs using different NTT configurations. The power results in Table V show a 6% to 14% higher power consumption in RoutOpt than in MemOpt. This is mainly caused by up to 11% higher clock frequency of RoutOpt. Considering the energy consumed per NTT, MemOpt requires up to 4% less energy than RoutOpt. Yet, for smaller parameters, the energy consumed per NTT is almost identical.

VI. COMPARISON TO RELATED WORKS

This section compares the benchmarks of OpenNTT's NTT and FFT designs to related works. We use matching parameter sets and identical FPGAs to allow fair comparisons. OpenNTT is a hardware compilation tool with on-the-fly TFG, whereas other NTT design tools mostly use stored TFs. To allow a direct comparison between these orthogonal design strategies, we use the area-time product $ATP = (LUT + 100 \cdot DSP + 300 \cdot BRAM) \cdot Lat$, as defined in [28]. OpenNTT produces either MemOpt or RoutOpt designs. We use MemOpt designs as a default option for comparison. RoutOpt performs best in larger designs occupying a significant portion of FPGA resources. Hence, we provide RoutOpt results for such large and congested designs.

A. Comparison to NTT Designs With Stored Twiddle Factors

Table VI presents a comparison to prior works with stored TFs targeting PQC and FHE use cases. The table shows the chosen parameter set, the used FPGA platform, the reported implementation results, and the ATP for each configuration. In addition, the transformation latency and ATP improvement achieved by OpenNTT are given. Note that all comparisons are based on identical FPGAs. We first compare to [14],

TABLE V
RESULTS FOR ROUTOPT/MEMOPT NTT DESIGNS ON ZCU102

N , n_{PE}	Freq. MHz	Latency cc	Latency μs	Power W	Energy μJ per NTT
2^{12} , 4	499/448	6,189/6,218	12.4/13.9	4.3/3.8	53/53
2^{13} , 8	468/437	6,698/6,727	14.3/15.4	7.2/6.8	103/105
2^{14} , 16	411/396	7,210/7,239	17.5/18.3	13.7/12.7	241/233
2^{16} , 32	295/270	16,426/16,455	55.7/60.9	23.5/20.7	1,309/1,259

which presents a conflict-free memory access pattern for different PE structures. Moreover, Mu et al. [14] used stored TFs, causing up to $2\times$ higher BRAM consumption than in OpenNTT's MemOpt configuration. Yet, OpenNTT's DSP consumption is approximately $2\times$ higher due to the additional multipliers for TFG. Our OpenNTT design reaches higher clock frequencies for PQC and FHE parameters due to optimized pipelining. This allows up to $2\times$ and $1.82\times$ performance improvement and ATP reduction, respectively, compared to [14]. Hu et al. [15] proposed an area-efficient NTT architecture with runtime flexible polynomial degrees N . Compared to [15] using stored TFs, OpenNTT's MemOpt designs reach up to $1.4\times$ and $1.7\times$ lower latency for PQC and FHE parameters, respectively. Considering BRAM consumption, OpenNTT's MemOpt designs instantiate up to $2.6\times$ fewer BRAMs than [15] due to the TFG in OpenNTT. When considering OpenNTT's RoutOpt design, a higher clock frequency of 259 MHz is achieved. Yet, RoutOpt needs more BRAMs compared to MemOpt due to the independent TFG modules. However, the BRAMs in RoutOpt are just partly filled; just 118 out of 512 memory locations are used. This means that up to four primes can be supported without increasing OpenNTT's BRAM consumption in this case. This article [12] presents a constant geometry out-of-place NTT hardware architecture. The out-of-place approach allows a uniform control flow while requiring a larger memory consumption. In contrast, OpenNTT uses in-place NTTs and TFG, thereby lowering the memory consumption by up to $2.4\times$ in MemOpt. At the same time, OpenNTT uses just 5% to 14% more DSPs than [12]. Thus, OpenNTT reaches a $1.24\times$ improved ATP. Our RoutOpt design further reduces the NTT latency by $1.8\times$ and $1.2\times$ compared to [12] and our MemOpt design, respectively. The recent paper AutoNTT [13] presents an HLS tool for automatic NTT design space exploration and supports iterative, pipelined, and hybrid architectures using stored TFs. Compared to iterative AutoNTT, our MemOpt design reaches similar latency and ATP, while RoutOpt shows a 20% latency reduction. Chen et al. [29] proposed a radix-flexible hardware design relying on stored TFs. Using the same n_{PE} as [29], OpenNTT's MemOpt reaches $1.1\times$ to $1.2\times$ lower latency. Yet, [29] has a better ATP for 2×2 PEs due to their low area consumption, whereas OpenNTT has a $1.23\times$ lower ATP than [29] for eight PEs. Cheng et al. [30] introduced optimizations for the Montgomery multiplication and design a 2×2 PE array for NTT. Their design uses stored TFs, which results in a $2.7\times$ higher memory consumption than in OpenNTT. Nevertheless, [30] reaches a lower ATP for $N = 2^{10}$ while OpenNTT has a lower ATP for $N = 2^{12}$, showing the benefits of our TFG in large-degree NTTs. The latency is similar in OpenNTT and [30]. Wang et al. [31] presented an

TABLE VI
COMPARISON WITH NTT HARDWARE ACCELERATORS USING STORED TFS

Work	Platform	Application	N	$\log q$	n_{PE}	LUT / FF / BRAM / DSP	Freq. MHz	Latency cc	NTT μs (impr.)	ATP ^a (impr.)
[14]	Virtex-7 ¹	PQC	256	24	1	691 / 451 / 3 / 5	187	1,031	5.5	12
		FHE	4096	24	1	802 / 525 / 7 / 4	185	24,583	132.9	439
		FHE	4096	24	8	5,665 / 3,188 / 8.5 / 33	157	3,079	19.7	227
Our MemOpt	Virtex-7 ¹	PQC	256	24	1	1,169 / 1,890 / 1.5 / 10	350	1,060	3.0 (1.8)	8 (1.45)
		FHE	4096	24	1	1,254 / 1,927 / 4 / 8	333	24,610	73.9 (1.8)	240 (1.82)
		FHE	4096	24	8	8,630 / 9,963 / 5 / 64	320	3,106	9.7 (2.0)	160 (1.41)
[15]	ZCU102 ²	PQC	256	24	2	2700 / 2200 / 5 / 8	333	526	1.58	8
	Artix-7 ³	PQC	256	24	1	1,300 / 1,100 / 2.5 / 4	152	1,038	6.85	17
	Virtex-7 ⁴	FHE	4096	32	4	6,300 / 5,200 / 14 / 24	224	6,158	27.46	354
		FHE	4096	60	1	2,600 / 2,500 / 21 / 26	144	24,590	171.6	1,973
		FHE	4096	60	8	22,100 / 19,500 / 48 / 208	141	3,086	21.91	1,255
Our MemOpt	ZCU102 ²	PQC	256	24	2	1,989 / 3,202 / 3 / 20	450	548	1.2 (1.3)	6 (1.33)
	Artix-7 ³	PQC	256	24	1	1,128 / 1,579 / 2 / 10	220	1,073	4.9 (1.4)	13 (1.26)
	Virtex-7 ⁴	FHE	4096	32	4	5,973 / 8,030 / 9 / 56	290	6,194	21.4 (1.3)	305 (1.16)
		FHE	4096	60	1	5,996 / 9,845 / 16 / 44	250	24,650	98.6 (1.7)	1,498 (1.32)
		FHE	4096	60	8	34,886 / 45,316 / 18 / 352	225	3,146	14.0 (1.6)	1,055 (1.19)
Our RoutOpt	Virtex-7 ⁴	FHE	4096	60	8	33,196 / 41,715 / 32 / 352	259	3,117	12.0 (1.8)	939 (1.34)
[12]	Virtex-7 ⁵	PQC	256	13	1	632 / 402 / 2.5 / 3	335	1,033	3.1	5.2
		FHE	4096	24	16	10,800 / 9,500 / 40 / 112	220	1,545	7.0	238
		FHE	4096	60	1	1,900 / 1,800 / 17 / 42	154	24,585	159.6	1,788
		FHE	4096	60	8	14,100 / 12,500 / 41 / 336	150	3,081	20.5	1,230
Our MemOpt	Virtex-7 ⁵	PQC	256	13	1	577 / 723 / 2 / 6	300	1,056	3.5 (0.9)	6.3 (0.83)
		FHE	4096	24	16	19,912 / 19,583 / 17 / 128	290	1,570	5.4 (1.3)	205 (1.16)
		FHE	4096	60	1	6,003 / 9,600 / 16 / 44	250	24,650	98.6 (1.6)	1,499 (1.19)
		FHE	4096	60	8	35,262 / 44,107 / 18 / 352	240	3,146	13.1 (1.6)	994 (1.24)
Our RoutOpt	Virtex-7 ⁵	FHE	4096	60	8	33,380 / 41,760 / 32 / 352	278	3,117	11.2 (1.8)	877 (1.40)
[13]	U50 ⁸	FHE	65536	52	32	80,000 / 60,000 / 498 / 288	250	16,635	66.5	17,181
Our MemOpt	U50 ⁸	FHE	65536	52	32	158,059 / 117,392 / 98 / 896	250	16,455	65.8 (1.0)	18,236 (0.94)
Our RoutOpt	U50 ⁸	FHE	65536	52	32	156,138 / 116,354 / 160 / 896	295	16,426	55.7 (1.2)	16,356 (1.05)
[29]	Virtex-7 ⁵	FHE	1024	14	2×2	1,196 / 969 / 3 / 12	270	1,308	4.8	16
		FHE	1024	14	8	6,300 / 2,124 / 10 / 27	227	654	2.9	35
Our MemOpt	Virtex-7 ⁵	FHE	1024	14	4	2,167 / 2,640 / 5 / 24	290	1,327	4.6 (1.1)	28 (0.57)
		FHE	1024	14	8	4,431 / 4,948 / 9 / 48	290	686	2.4 (1.2)	28 (1.23)
[30]	Virtex-7 ⁷	FHE	1024	32	2×2	4,655 / 4,327 / 9 / 16	333	1,310	3.9	35
		FHE	4096	32	2×2	4,747 / 4,034 / 24 / 16	297	6,168	20.8	281
Our MemOpt	Virtex-7 ⁷	FHE	1024	32	4	6,082 / 8,250 / 5 / 56	333	1,330	4.0 (1.0)	53 (0.67)
		FHE	4096	32	4	6,236 / 8,626 / 9 / 56	340	6,194	18.2 (1.1)	265 (1.06)
[31]	Virtex-7 ¹	FHE	4096	32	4	13,630 / 14,441 / 18.5 / 0	294	6,195	21.1	404
Our MemOpt	Virtex-7 ¹	FHE	4096	32	4	6,209 / 8,496 / 9 / 56	310	6,194	20.0 (1.1)	290 (1.39)
[32]	Virtex-7 ¹	FHE	4096	60	12	11,856 / 11,222 / 23.5 / 96	183	2,069	11.3	322
Our MemOpt	Virtex-7 ¹	FHE	4096	60	8	35,976 / 44,942 / 18 / 352	230	3,146	13.7 (0.8)	1,047 (0.31)
Our RoutOpt	Virtex-7 ¹	FHE	4096	60	8	32,158 / 39,458 / 32 / 352	310	3,117	10.1 (1.1)	774 (0.42)
[18] ^{SDF}	Virtex-7 ⁶	FHE	1024	32	10	5,200 / 2,800 / 2 / 36	150	2.1k/1.0k ^b	14.1/6.9 ^b	133/65 ^b
[18] ^{MDC}	Virtex-7 ⁶	FHE	1024	32	10	6,900 / 5,200 / 2 / 36	150	1.1k/0.5k ^b	7.3/3.5 ^b	81/39 ^b
Our MemOpt	Virtex-7 ⁶	FHE	1024	32	8	11,859 / 14,046 / 9 / 112	270	667	2.5(1.4/5.4)	65 (0.6/2)

^a: Area-time product; ^{SDF}: pipelined with SDF; ^{MDC}: pipelined with MDC; ^b: single / average NTT; ¹: xc7vx690tffg1761-2; ²: xczu9eg-ffvb1156-2-e;

³: xc7a200tfbg676-2; ⁴: xc7vx485tffg1761-2; ⁵: xc7vx690tffg1761-3; ⁶: xc7vx485tffg1157-2; ⁷: xc7v585tffg1157-3; ⁸: xcu50-fsvh2104-2-e

in-place constant-geometry NTT dedicated to radix-4 PEs. The presented technique uses stored TFs and five memory banks to store the polynomial. Compared to [31] using LUT-based multipliers, OpenNTT reaches $1.1\times$ and $1.39\times$ lower latency and ATP, respectively. Next, we consider [32], presenting an optimized memory access scheme within a pipelined NTT architecture. Wang et al. [32] used stored TFs, which ease the NTT control flow. As reported in Table VI, [32] instantiates 12 PEs and reaches a lower logic consumption than OpenNTT with eight PEs. This is due to our TFG and the Shoup reduction algorithm used in [32]. Nevertheless, when scaled to the same n_{PE} , OpenNTT reaches $1.2\times$ and $1.6\times$ better performance for MemOpt and RoutOpt, respectively, while the ATP is higher than in [32]. As the last NTT work with stored TFs, we consider [18]. Unlike OpenNTT, which compiles iterative NTT designs, Hirner et al. [18] proposed pipelined

NTT designs using SDF and MDC. Pipelined NTT designs allow higher throughputs when multiple sequential NTTs need to be performed. Therefore, Table VI shows two latency values for single NTT latency/average NTT latency. Compared to [18], OpenNTT reaches a latency reduction between $1.4\times$ and $5.4\times$, although using fewer PEs. However, the resource utilization of OpenNTT is significantly higher than in [18], leading to a worse ATP when considering an average case.

B. Comparison to NTT Designs With TFG

We now compare OpenNTT to related works that use TFG. Table VII shows the according benchmarks. Duong-Ngoc et al. [33] presented an NTT accelerator dedicated to FHE applications. The accelerator supports FHE-typical large polynomials with $N = 2^{16}$ and $n_{PE} = 32$ PEs. Since [33] and

TABLE VII
COMPARISON TO NTT HARDWARE ACCELERATORS WITH GENERATED TFS AND FFT DESIGNS

Work	Platform	Appli- cation	N	$\log q$ / Precision	n_{PE}	LUT / FF / BRAM / DSP	Freq. MHz	Latency cc	NTT / FFT μs (impr.)	ATP ^a (impr.)
NTT Designs with Generated TF										
[33]	ZCU102 ²	FHE	65536	52	32	149k / 91k / 137 / 564	200	16,776	83.9	20,675
Our MemOpt	ZCU102 ²	FHE	65536	52	32	157k / 117k / 98 / 896	210	16,455	78.4 (1.1)	21,653 (0.95)
Our RoutOpt	ZCU102 ²	FHE	65536	52	32	156k / 116k / 160 / 896	265	16,426	62.0 (1.4)	18,191 (1.14)
[34]	Virtex-7 ⁵	FHE	32768	32	32	36k / 51k / 88 / 392	240	337,272	1,405.3	142,216
Our MemOpt	Virtex-7 ⁵	FHE	32768	32	32	73k / 55k / 65 / 384	200	7,728	38.6 (36)	5,073 (28)
[35]	U250 ⁹	FHE	16384	54	16	76k / 57k / 496 / 360 ^b	200	7,200	36.0	9,406
Our MemOpt	U250 ⁹	FHE	16384	54	16	71k / 71k / 50 / 544	250	7,239	29.0 (1.2)	4,059 (2.32)
Our RoutOpt	U250 ⁹	FHE	16384	54	16	69k / 69k / 80 / 544	290	7,210	24.9 (1.4)	3,667 (2.57)
FFT Designs										
[8] (TFG)	Kintex-7 ⁷	FHE	8192	double	1	20,728 / 17,647 / 82.5 / 100	200	53,400	267	14,813
Our MemOpt	Kintex-7 ⁷	FHE	8192	double	1	15,086 / 14,607 / 33 / 88	235	53,292	226.8 (1.2)	7,662 (1.93)
[36] (stored TFs)	ZCU104 ¹⁰	PQC	256	double	1	8,396 / 2,526 / 9.5 / 9	200	10,236	51.2	622
		PQC	512	double	1	8,396 / 2,526 / 9.5 / 9	200	19,800	99	1,202
Our MemOpt	ZCU104 ¹⁰	PQC	256	double	1	14,922 / 13,838 / 8 / 88	333	1,068	3.2 (15.9)	84 (7.42)
		PQC	512	double	1	15,027 / 13,819 / 8 / 88	333	2,355	7.1 (14.0)	185 (6.48)

^a: Area-time product; ^b: One RPAU; ²: xczu9eg-ffvb1156-2-e; ⁵: xc7vx690tffg1761-3; ⁷: xc7k325tffg900-2; ⁹: xcuz250-figd2104-2L-e; ¹⁰: xczu7ev-ffvc1156-2-e

OpenNTT use TFG, the results are similar for our MemOpt configuration; MemOpt shows 7% lower latency but 5% higher ATP. OpenNTT's RoutOpt shows a latency reduction by $1.4\times$ and $1.3\times$ compared to [33] and our MemOpt design, respectively. Yet, this speedup demands a higher BRAM consumption, leading to a slightly lower ATP improvement of $1.14\times$ concerning [33]. Huang et al. [34] targeted large-scale FHE parameter sets while reducing area consumption. Their constant-geometry NTT methodology requires stall cycles between transformation stages to flush the pipeline. In addition, [34] only instantiates one TwGen module for two PEs, which lowers the area consumption but increases latency. This explains the $36\times/28\times$ performance/ATP improvement of OpenNTT over [34] and underlines the benefit of stall-free transformations. Another work using TFG is [35], which is a full-fledged FHE accelerator with high BRAM consumption for key storage. This causes an up to $2.57\times$ and $1.4\times$ higher ATP and latency, respectively, in [35] than in OpenNTT's MemOpt and RoutOpt designs.

C. Comparison to FFT Designs

Finally, we compare OpenNTT's FFT designs to related works; the according benchmarks are shown in Table VII (bottom). Krieger et al. [8] designed an FHE accelerator for client-side operations in the CKKS scheme [24]. Therein, an FFT transformation is needed, which takes $267\mu s$. OpenNTT reduces the latency for the same parameter set by $1.2\times$. A comparison of resource consumption and ATP is hard since [8] is an end-to-end FHE accelerator while OpenNTT just provides FFT designs. Nevertheless, OpenNTT lowers memory consumption by $2.5\times$ and LUT consumption by 40%. Mandal and Roy [36] presented a hardware-software co-design for accelerating the FFT in the Falcon PQ signature scheme. Mandal and Roy [36] instantiated one floating point adder and multiplier in hardware, which is iterated several times for one complex multiplication. This explains the low area consumption. Yet, compared to [36], OpenNTT reaches $15.9\times$ and $14\times$ lower FFT latency for $N = 256$ and $N = 512$, respectively. This benefit of OpenNTT is due

to the $1.7\times$ higher frequency achieved and due to the pure hardware implementation of FFT, whereas [36] follows a hardware-software co-design approach. OpenNTT also shows a moderate reduction in memory consumption and a slight increase in logic consumption, therefore, yielding an ATP improvement of up to $7.42\times$.

VII. DISCUSSION

A. Other Twiddle Factor Generation Approaches

Although existing works propose on-the-fly TFG architectures [8], [33], [34], [35], most of them are dedicated to one or a few parameter sets and a single transformation type. Generalizing these designs to any configuration and presenting a unified design tool with TFG is challenging.

For example, Duong-Ngoc et al. [33] presented a TFG method for 8×4 PE arrays, which require 15 TFs in each cycle. They follow a similar approach to OpenNTT and instantiate 15 TwGen modules, each delivering one TF sequence. Their TwGen modules fill the multiplier pipelines using a TF ROM, which requires storing $15\times 25 = 375$ TFs. This is $5\times$ more than our MemOpt TFG storing 75 TFs for the same configuration. In addition, [33] packs TFs for all PEs into a single centralized memory, whereas our RoutOpt strategy allows *distributed* placement of the TwROM to improve the frequency by 32.5% over [33] (see Table VII). Yet, our RoutOpt needs 4.8-k TFs in TwROM, which is more than in [33]. Note that [33] only uses modular multipliers with 21-cc latency and $N = 2^{16}$ -point DIT forward and DIF inverse NTTs. Duong-Ngoc et al. [33] does not discuss how the presented methodology can be extended to other parameter sets, while OpenNTT directly supports a wide range of configurations.

Huang et al. [34] proposed an out-of-place constant-geometry NTT with TFG and a variable number of PEs. Constant geometry NTT follows the same memory access pattern in each stage, which simplifies the processing order. Moreover, [34] only instantiates $\lceil (n_{PE}/2) \rceil$ TFG modules to save area at the cost of pipeline stalls and increased latency. Huang et al. [34] stored $2(\log_2(N) + 1) \lceil (n_{PE}/2) \rceil$ TFs in ROM per modulus, whereas MemOpt, also reducing resource

consumption, stores $1 + n_{PE} + d_{Mul}$ TF while allowing a fully pipelined design.

Another interesting work is [37], which proposes a hybrid solution between stored and on-the-fly generated TFs. [37] splits each NTT TF into two chunks and stores them separately in memory. The chunks are combined during runtime, yielding the desired TF sequence. This reduces the ROM demand but still has a linear dependency on the polynomial degree, unlike in OpenNTT. The open-source work in [8] uses TFG but requires pipeline stalls between transformation stages. This increases the latency of NTT and FFT, which is not the case in OpenNTT thanks to our fully pipelined TFG and conflict-free memory accesses. Other works [35] do not detail their TFG architecture, making a comparison of methodology hard.

Overall, our generic methodology of on-the-fly TFG complements existing works. In addition, OpenNTT extends to stall-free transformations for any relevant NTT and FFT configuration within a unified hardware generation tool.

B. Other Conflict-Free Memory Access Strategies

There exist different approaches to conflict-free memory access patterns in the literature. One of these efforts is [29] proposing a conflict-free memory access pattern for multi-PE and higher radix NTTs. However, while ensuring conflict-free memory accesses, the processing order in [29, Alg. 4] requires TFs in bit-reversed order. For example, in the last stage of a radix-4, $N = 256$ -point DIT NR NTT, [29] needs a nonlinear TF sequence of $\omega^1, \omega^{65}, \omega^{33}, \omega^{97}, \dots$ which is hard to compute on the fly. Thus, [29] precomputes and stores all TFs in random-access ROM, thereby allowing straightforward access to any required TF. Similarly, the work in [30] improves the radix-4 processing flow compared to [29]. Yet, [30] also stores TFs in ROM, which relaxes the demands on processing order and memory access pattern. The work in [14] introduces a generic processing flow for varying PE array structures (including single PE and radix-4 designs). Also [14] stores TFs in ROM, which avoids special handling of the TF generation.

Similarly, [31] uses an in-place constant-geometry NTT which enhances uniformity of memory accesses across stages. [31] also keeps the TFs in ROM, thereby simplifying the control flow constraints. The work [32] targets a pipelined NTT design and presents an optimized memory access pattern. Nevertheless, [32] uses stored TFs, and the control flow for pipelined NTT do not directly apply to our iterative approach.

In contrast to [14], [29], [30], [31], and [32], where ROM lookups easily obtain any TF, our design offers on-the-fly TFG. This significantly reduces the memory demand but introduces additional constraints on the processing order, which is not the case when using stored TF with ad-hoc loading from random-access ROM. Our novel processing order (discussed in Section III-A and detailed in [16]) obeys these constraints and ensures an efficient TFG pattern which contrasts with the TF sequence in [14], [29], and [30]. In addition, [29] and [30] only target merged DIT NR forward and DIF RN inverse NTTs. Yet, our unified NR and RN flows in Section III readily support all relevant NR/RN, DIT/DIF, and plain/merged NTT/FFT combinations.

C. Applicability to Higher Radices

Our current OpenNTT design supports a variable number of radix-2 PEs. Yet, extending our MemOpt and RoutOpt concepts to higher radices is possible. For example, in radix-4, three TFs per PE are required in each cycle. These TFs can be computed by instantiating three TwGen modules, each serving one TF sequence. Extending our conflict-free yet TFG-friendly processing order to high radix designs is another interesting topic, which we leave open for future investigations.

D. Side-Channel Security

The NTT and FFT algorithms and TFs are public. Since our RoutOpt and MemOpt strategies compute the public TF sequence in a secret-independent manner, our methodology does not introduce timing side-channels. However, mitigating power side channels is beyond the scope of this work. Nevertheless, techniques like arithmetic masking may instantiate multiple OpenNTT-generated modules, where each instance processes one independent share. Thus, OpenNTT also supports the design process of masked hardware.

VIII. CONCLUSION

In this article, we proposed an extended version of our OpenNTT tool that generates efficient FFT and NTT hardware accelerators with on-the-fly TFG. OpenNTT combines efficient TFG and conflict-free memory accesses through our generic processing order. Moreover, we presented the RoutOpt and MemOpt optimizations, where MemOpt effectively reduces the ROM memory consumption and RoutOpt trades slightly larger memories for higher clock frequencies. The comparison to other NTT and FFT designs showed competitive results and speedups in ATP and latency, respectively.

REFERENCES

- [1] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. ASIACRYPT*, 2017, pp. 409–437.
- [2] S. Halevi, Y. Polyakov, and V. Shoup, "An improved RNS variant of the BFV homomorphic encryption scheme," in *Proc. Cryptogr. Track RSA Conf.*, 2019, pp. 83–105.
- [3] J. Bos et al., "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE Eur. Symp. Security Privacy*, 2018, pp. 353–367.
- [4] P.-A. Fouque et al., "Falcon—Fast-fourier lattice-based compact signatures over NTRU." Accessed: Dec. 2024. [Online]. Available: <https://falcon-sign.info/>
- [5] L. Ducas et al., "Crystals-dilithium: A lattice-based digital signature scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.
- [6] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Proc. Int. Conf. Cryptol. Netw. Security*, 2016, pp. 124–139.
- [7] V. Lyubashevsky, D. Micciancio, C. Peikert, and A. Rosen, "SWIFT: A modest proposal for FFT hashing," in *Proc. Int. Workshop Fast Softw. Encrypt.*, 2008, pp. 54–72.
- [8] F. Krieger, F. Hirner, A. C. Mert, and S. S. Roy, "Aloha-HE: A low-area hardware accelerator for client-side operations in homomorphic encryption," in *Proc. Design, Autom. Test Europe Conf. Exhib.*, 2024, pp. 1–6.
- [9] R. Paludo and L. Sousa, "NTT architecture for a linux-ready RISC-V fully-homomorphic encryption accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 7, pp. 2669–2682, Jul. 2022.
- [10] N. Zhang, B. Yang, C. Chen, S. Yin, S. Wei, and L. Liu, "Highly efficient architecture of NewHope-NIST on FPGA using low-complexity NTT/INTT," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 2, pp. 49–72, 2020.

- [11] A. C. Mert, E. Karabulut, E. Öztürk, E. Savaş, M. Becchi, and A. Aysu, "A flexible and scalable NTT hardware: Applications from homomorphically encrypted deep learning to post-quantum cryptography," in *Proc. Design, Autom. Test Europe Conf. Exhib.*, 2020, pp. 346–351.
- [12] S.-H. Liu, C.-Y. Kuo, Y.-N. Mo, and T. Su, "An area-efficient, conflict-free, and configurable architecture for accelerating NTT/INTT," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 32, no. 3, pp. 519–529, Mar. 2024.
- [13] D. Kumarathunga, Q. Hu, and Z. Fang, "AutoNTT: Automatic architecture design and exploration for number theoretic transform acceleration on FPGAs," in *Proc. IEEE 33rd Annu. Int. Symp. Field-Programm. Custom Comput. Mach.*, 2025, pp. 1–9.
- [14] J. Mu et al., "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof, and implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 42, no. 5, pp. 1504–1517, May 2023.
- [15] X. Hu, J. Tian, M. Li, and Z. Wang, "AC-PM: An area-efficient and configurable polynomial multiplier for lattice based cryptography," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 719–732, Feb. 2023.
- [16] F. Krieger, F. Hirner, A. C. Mert, and S. Sinha Roy, "OpenNTT—An automated toolchain for compiling high-performance NTT accelerators in FHE," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Design*, 2025, pp. 1–9.
- [17] "OpenNTT source code." Accessed: Aug. 3, 2025. [Online]. Available: <https://github.com/flokrieger/OpenNTT>
- [18] F. Hirner, A. C. Mert, and S. S. Roy, "Proteus: A pipelined NTT architecture generator," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 32, no. 7, pp. 1228–1238, Jul. 2024.
- [19] N. Zhang et al., "NTTU: An area-efficient low-power NTT-uncoupled architecture for NTT-based multiplication," *IEEE Trans. Comput.*, vol. 69, no. 4, pp. 520–533, Apr. 2020.
- [20] (Nat. Inst. Stand. Technol., Gaithersburg, MA, USA). *Announcement of Quantum-Resistant Cryptographic Algorithms*. 2022. [Online]. Available: <https://www.nist.gov/news-events/news/2022/07/nist-announces-first-four-quantum-resistant-cryptographic-algorithms>
- [21] T. Pöppelmann, T. Oder, and T. Güneysu, "High-performance ideal lattice-based cryptography on 8-bit ATxmega Microcontrollers," *Cryptol. ePrint Arch.*, IACR, Bellevue, WA, USA, Rep. 2015/382, 2015.
- [22] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, "Compact ring-LWE cryptoprocessor," in *Proc. Int. Workshop Trans. Cryptogr. Hardw. Embed. Syst.*, 2014, pp. 371–391.
- [23] (Microsoft, Redmond, WA, USA). *Microsoft SEAL (Release 4.1)*. Jan. 2023. [Online]. Available: <https://github.com/Microsoft/SEAL>
- [24] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full RNS variant of approximate homomorphic encryption," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2019, pp. 347–368.
- [25] (AMD Inc., Santa Clara, CA, USA). *MicroBlaze Processor Reference Guide (UG984)*. Nov. 2024. [Online]. Available: <https://docs.amd.com/r/en-US/ug984-vivado-microblaze-ref?tocId=kapC3do5RYcn10xvXtz0Ig>
- [26] (AMD Inc., Santa Clara, CA, USA). *AMD Zynq 7000 SoCs*. Accessed: Apr. 30, 2025. [Online]. Available: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html>
- [27] M. Albrecht et al., "Homomorphic encryption standard," *Cryptol. ePrint Arch.*, IACR, Bellevue, WA, USA, Rep. 2019/939, 2019.
- [28] Z. Ye, R. C. C. Cheung, and K. Huang, "PipeNTT: A pipelined number theoretic transform architecture," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 69, no. 10, pp. 4068–4072, Oct. 2022.
- [29] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 94–126, Nov. 2021.
- [30] Z. Cheng, B. Zhang, and M. Pedram, "A high-performance, conflict-free memory-access architecture for modular polynomial multiplication," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 2, pp. 492–505, Feb. 2024.
- [31] J. Wang et al., "A reconfigurable and area-efficient polynomial multiplier using a novel in-place constant-geometry NTT/INTT and conflict-free memory mapping scheme," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 3, pp. 1358–1371, Mar. 2025.
- [32] Z. Wang, Y. Yang, J. Wang, J. Hou, Y. Su, and C. Yang, "A scalable and efficient NTT/INTT architecture using group-based pairwise memory access and fast interstage reordering," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 33, no. 2, pp. 588–592, Feb. 2025.
- [33] P. Duong-Ngoc, S. Kwon, D. Yoo, and H. Lee, "Area-efficient number theoretic transform architecture for homomorphic encryption," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 3, pp. 1270–1283, Mar. 2023.
- [34] J. Huang, C. Kuo, S. Liu, and T. Su, "An area-efficient and configurable number theoretic transform accelerator for homomorphic encryption," *Electronics*, vol. 13, no. 17, p. 3382, 2024.
- [35] A. C. Mert et al., "Medha: Microcoded hardware accelerator for computing on encrypted data," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 1, pp. 463–500, 2022.
- [36] S. Mandal and D. Basu Roy, "Design of a lightweight fast fourier transformation for FALCON using hardware-software co-design," in *Proc. Great Lakes Symp. VLSI*, 2024, pp. 228–232.
- [37] S. Kim et al., "BTS: An accelerator for bootstrappable fully homomorphic encryption," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, 2022, pp. 711–725.



Florian Krieger received the B.Sc. and M.Sc. degrees from the Graz University of Technology, Graz, Austria, in 2022 and 2023, respectively, where he is currently pursuing the Ph.D. degree with the Institute of Information Security.

His research interests are the design aspects of PQC, FHE, and ZKP.



Florian Hirner received the B.Sc. and M.Sc. degrees in computer science from the Graz University of Technology, Graz, Austria, in 2020 and 2022, respectively, where he is currently pursuing the Ph.D. degree with the Institute of Information Security.

His research interests include PQC, ZKP, and hardware design.



Ahmet Can Mert received the Ph.D. degree in electronics engineering from Sabanci University, Istanbul, Türkiye, in 2021.

From 2021 to 2024, he worked as a Postdoctoral Researcher with the Institute of Information Security, Graz University of Technology, Graz, Austria. His research interests include homomorphic encryption, lattice-based cryptography, and hardware design.



Sujoy Sinha Roy (Senior Member, IEEE) received the Ph.D. degree from KU Leuven, Leuven, Belgium, in 2017.

He is currently an Associate Professor of cryptographic engineering with the Institute of Information Security, Graz University of Technology, Graz, Austria. He is a Co-Designer of "Saber," which is a finalist key encapsulation mechanism (KEM) candidate in NIST's Post-Quantum Cryptography Standardization Project. He is interested in the implementation aspects of cryptography.